

Automotive Software and Standards Guide

1. Introduction

Modern vehicles contain many electronic control units (ECUs) and millions of lines of software. Because software controls braking, steering, airbags and driver assistance, the industry follows common standards for architecture, functional safety, cybersecurity, process quality and diagnostics. This guide gives a short overview of the most important ones.

2. AUTOSAR - Overview

AUTOSAR stands for AUTomotive Open System ARchitecture. It is a worldwide partnership of vehicle manufacturers, suppliers and tool companies, started in 2003. Its goal is a standard software architecture so that software components can be reused across different vehicles and ECUs from different suppliers. AUTOSAR has two main platforms: the Classic Platform and the Adaptive Platform. A third part, the Foundation, defines shared specifications used by both.

3. AUTOSAR Classic Platform

The Classic Platform is designed for deeply embedded, real-time ECUs such as engine control, braking and airbag controllers. It uses a layered architecture. At the top is the Application Layer, which contains Software Components (SWCs). Below it is the Runtime Environment (RTE), which connects the software components to each other and to the basic software. The Basic Software (BSW) contains the Services Layer, the ECU Abstraction Layer, the Microcontroller Abstraction Layer and Complex Drivers. At the bottom is the Microcontroller Abstraction Layer (MCAL), which talks directly to the hardware. The operating system in the Classic Platform is based on OSEK/VDX and is statically configured. Configuration is described in ARXML files.

4. AUTOSAR Adaptive Platform

The Adaptive Platform is designed for high-performance computers in the vehicle, for example for autonomous driving, sensor fusion, connectivity and over-the-air (OTA) updates. It runs on a POSIX-based operating system and is written in modern C++. It uses service-oriented communication, mainly SOME/IP, through the ara::com API. Important functional clusters are Execution Management, State Management, Communication Management, Update and Configuration Management, Diagnostics, Persistency, Cryptography, Identity and Access Management, and Log and Trace. Unlike the Classic Platform, applications can be started, stopped and updated dynamically while the vehicle is in use.

5. ISO 26262 - Functional Safety

ISO 26262, titled Road vehicles - Functional safety, is the functional safety standard for electrical and electronic systems in passenger cars. It is derived from IEC 61508. The second edition was published in 2018 and has 12 parts. Functional safety means avoiding unreasonable risk caused by malfunctioning behaviour of E/E systems.

6. ISO 26262 - ASIL Levels

ISO 26262 defines the Automotive Safety Integrity Level (ASIL). There are four levels: ASIL A, ASIL B, ASIL C and ASIL D. ASIL A is the lowest and ASIL D is the highest (strictest) level. A function with no safety relevance is rated QM (Quality Management). The ASIL is found during the Hazard Analysis and Risk Assessment (HARA). Each hazardous event is rated by three factors: Severity (how bad is the injury),

Exposure (how often is the situation met) and Controllability (can the driver avoid the harm). The result is the ASIL and a safety goal for the system. Example: an unintended airbag deployment or a loss of braking is usually rated high, up to ASIL D.

7. ISO 26262 - Parts and Safety Process

Important parts are: Part 3 Concept phase, Part 4 Product development at the system level, Part 5 Product development at the hardware level, Part 6 Product development at the software level, Part 8 Supporting processes and Part 9 ASIL-oriented and safety-oriented analyses. The process follows the V-model: safety goals lead to a functional safety concept, then a technical safety concept, then hardware and software requirements, followed by verification and validation. ASIL decomposition allows a high ASIL requirement to be split into redundant, independent elements of lower ASIL, for example ASIL D can become ASIL B(D) + ASIL B(D).

8. ISO 26262 - Software Requirements (Part 6)

For software, ISO 26262 Part 6 asks for a safe architecture, use of coding guidelines such as MISRA C, unit testing, integration testing and structural coverage measurement. For ASIL D the standard strongly recommends MC/DC (Modified Condition/Decision Coverage). Other methods include static code analysis, freedom from interference between software parts of different ASIL, fault injection tests and requirements-based testing.

9. ISO 21448 - SOTIF

ISO 21448, called SOTIF (Safety Of The Intended Functionality), covers hazards that happen even when nothing is broken. They come from performance limitations or misuse, for example a camera that cannot see well in heavy fog or a machine-learning model that misclassifies an object. SOTIF is very important for ADAS and automated driving. It reduces the areas of unknown unsafe scenarios by analysis, simulation and test drives.

10. ISO/SAE 21434 - Cybersecurity

ISO/SAE 21434, Road vehicles - Cybersecurity engineering, was published in 2021. It defines cybersecurity activities across the whole vehicle lifecycle: concept, development, production, operation, maintenance and decommissioning. A central activity is the Threat Analysis and Risk Assessment (TARA), which identifies assets, threats, attack paths and risk levels. Results are cybersecurity goals and requirements.

11. UNECE R155 and R156

UNECE WP.29 regulation R155 requires vehicle manufacturers to run a Cyber Security Management System (CSMS). Regulation R156 requires a Software Update Management System (SUMS), which covers secure software updates including over-the-air updates. They are required for type approval in the EU, Japan, South Korea and other countries. They apply to new vehicle types from July 2022 and to all newly produced vehicles from July 2024. ISO/SAE 21434 is widely used to show compliance with R155.

12. Automotive SPICE (ASPICE)

Automotive SPICE is a process assessment model based on ISO/IEC 33020. It is used to judge how mature a supplier's development processes are. It has capability levels from 0 to 5: Level 0 Incomplete, Level 1 Performed, Level 2 Managed, Level 3 Established, Level 4 Predictable and Level 5 Innovating. Vehicle manufacturers usually ask suppliers for Level 2 or Level 3. The software engineering processes follow the V-model: SWE.1 Software Requirements Analysis, SWE.2 Software Architectural Design, SWE.3 Software

Detailed Design and Unit Construction, SWE.4 Software Unit Verification, SWE.5 Software Integration and Integration Test, and SWE.6 Software Qualification Test.

13. In-Vehicle Networks

CAN (Controller Area Network, ISO 11898) is the most common bus. Classical CAN reaches 1 Mbit/s with up to 8 data bytes per frame. CAN FD (Flexible Data-rate) allows up to 64 data bytes and a faster data phase, typically up to 5 Mbit/s or more. LIN (Local Interconnect Network) is a cheap, slow single-master bus with up to 20 kbit/s, used for mirrors, seats and window lifts. FlexRay offers 10 Mbit/s and deterministic timing. Automotive Ethernet (100BASE-T1 and 1000BASE-T1) is used for cameras, infotainment, high-speed data and software updates, and uses the SOME/IP protocol for services.

14. Diagnostics - UDS (ISO 14229)

Unified Diagnostic Services (UDS) is defined in ISO 14229. A diagnostic tester sends a request and the ECU gives a positive or negative response. Common services: 0x10 Diagnostic Session Control, 0x11 ECU Reset, 0x14 Clear Diagnostic Information, 0x19 Read DTC Information, 0x22 Read Data By Identifier, 0x27 Security Access, 0x2E Write Data By Identifier, 0x31 Routine Control, 0x34 Request Download, 0x36 Transfer Data, 0x37 Request Transfer Exit and 0x3E Tester Present. A negative response starts with 0x7F. UDS over CAN uses the transport protocol ISO 15765-2 (DoCAN), and UDS over Ethernet uses DoIP (ISO 13400). DTC means Diagnostic Trouble Code.

15. Software Testing in Automotive

Testing happens in several stages. MIL (Model-in-the-Loop) tests the model, SIL (Software-in-the-Loop) tests the compiled code on a PC, PIL (Processor-in-the-Loop) tests on the target processor, and HIL (Hardware-in-the-Loop) tests the real ECU connected to a simulated vehicle. Common coding rules are MISRA C and MISRA C++, which help to avoid unsafe language features. Static analysis, code reviews, unit tests and code coverage measurement are normal parts of the process.

16. Software-Defined Vehicle (SDV)

In a software-defined vehicle, features are mainly delivered by software and can be added or improved after sale through OTA updates. The electronic architecture moves from many small ECUs to a few powerful domain or zonal controllers connected by Ethernet. A hypervisor can run functions of different safety levels on the same hardware, for example a safety-critical instrument cluster and a non-critical infotainment system. Secure boot, signed updates and the Uptane framework help to protect OTA updates.